

Cloud Computing Applications and Services

Practical Assignment

2026/2027

General Information

- Each group consists of **5 members**. Registration is done through the *Fellowly* platform.
- The practical assignment has **3 distinct evaluation moments**: two intermediate *checkpoints* and a final submission with an in-person evaluation. These are described in the *Evaluation* Section.
- Note that the maximum score for the different stages of the assignment depends both on their correct implementation and on the quality of the report and each group's discussion with the teaching staff.
- A *FAQ* with answers to questions and other important information for carrying out the assignment will be maintained and updated in the “*Discussions*” section of the ASCN-UM organization, accessible to all students.
- Each group's repositories will also be hosted in this organization.
- Each group will have access to Google Cloud credits, provided by the teaching staff, to carry out the assignment.

Assignment Description

The practical assignment aims to exercise the knowledge acquired in the course in order to automate the installation, configuration, monitoring and evaluation of the **Warracker** application (<https://github.com/sassanix/warracker>). Briefly, Warracker is a web application that simplifies product warranty management.

Tasks

Task 1 – Automatic Installation and Configuration of the Application

Objective: The first task consists of using the **Ansible** tool to automate the installation and configuration of the Warracker application on the Google Cloud **Google Kubernetes Engine (GKE)** service.

Requirements (*i.e.*, in addition to using Ansible and the GKE service):

- The different components of the application should, whenever possible, run in separate *pods*.
- The installation and configuration of the application, as well as of the Google Cloud platform services, must be carried out automatically with the fewest possible manual steps.
- It must be possible to stop the application and resume it (e.g., due to scheduled maintenance) without losing critical data (e.g., user data).
- It is **mandatory** to respect the structure and functionality of the Ansible *playbooks* provided (see the *Development Requirements* Section).

Outcome: After installation and configuration, the Warracker application must be accessible from the outside (e.g., through a *browser* or HTTP requests).

Task 2 – Exploration and Optimization of the Application

Objective: The second task aims to understand and optimize the performance, scalability and resilience of the application. The group is expected to answer the following questions:

1. Considering the base installation proposed by the group for Task 1:
 - a. For an increasing number of clients, which application components may constitute a performance bottleneck?
 - b. What is the application's performance under different numbers of clients and workloads?
 - c. Which application components may constitute a single point of failure?
2. Based on the answers to the previous questions:
 - a. What load distribution/replication optimizations can be applied to the base installation?
 - b. What is the impact of the proposed optimizations on the application's performance and/or resilience?

Considerations:

- The previous questions must be discussed in the final report of the practical assignment, using the knowledge acquired in the course.
- The group must apply load distribution/replication mechanisms that optimize the scalability and/or resilience of the base installation.
Note: To reduce the complexity of this task, the group should focus on only a **single component** of the application (e.g., web/application server, database), justifying the choice.
- It is suggested that the group explore the monitoring tool provided by the Google Cloud platform and consider different monitoring metrics (e.g., CPU, RAM, I/O).
- Experimental tests should be considered to evaluate different features and/or components of the application, taking into account typical user interaction with the application.

The optimization, evaluation and monitoring solutions proposed in this task must be **included in the Ansible playbooks** so that they can be executed and reproduced automatically.

Evaluation

The practical assignment includes 3 evaluation phases, described below:

Checkpoint #1 (3 points): This *checkpoint* aims to validate the group's knowledge of the **Warracker** application, namely its architecture, functionality and components. Additionally, this *checkpoint* will serve to validate the group's initial plan for approaching the tasks of the practical assignment.

Deadline: November 8, 2026

Considerations:

- This *checkpoint*, submitted in the form of an initial report, will be evaluated by the teaching staff.
- The requirements for this initial report are described in the [Initial Report Requirements](#) section.
- The report must be submitted through the group's Git repository, in the *ASCN-UM* organization, by **November 8, 2026, at 23:59**; the version corresponding to the last *commit* made up to the

indicated date and time will be considered for evaluation purposes.

Checkpoint #2 (5 points): This *checkpoint* aims to validate that, once installed and configured on the Google Cloud platform, the **Warracker** application works correctly and is accessible from the outside.

Deadline: December 6, 2026

Considerations:

- This *checkpoint* will be evaluated automatically, through the group's **GitHub repository**.
- No report submission is required for this *checkpoint*.

Final Evaluation (12 points): The final evaluation will consider all components of the practical assignment that were not evaluated in the *checkpoints*.

Deadline: January 30, 2026

Considerations:

- The practical assignment is completed with the submission of the **final report and all the files that make up the developed solution**, through the group's GitHub repository, in the *ASCN-UM* organization. The repository must include all *scripts*, configuration files and other resources necessary to run and reproduce the solution.
- It is **mandatory** to identify, on the report cover, the group number and the names of all its members.
- The deadline for submitting the assignment is **December 30, 2026**. After this date, **no reports or new commits** in the GitHub repository will be considered.
- The presentation and discussion of the assignment will take place during the week of January 11 to 16, 2027.

Initial Report Requirements

The report submitted must contain the following information. Note that the order in which each of the points is presented in the report is left to the group's discretion.

- A brief description of the architecture and main components of the **Warracker** application.
- A brief description of the features and APIs provided by the application.
- A theoretical discussion (practical validation is left for the final report) of potential performance bottlenecks and single points of failure in the application's installation.
- A reflection on open questions in the previous plan and identification of difficulties in installing the application. This installation may be done manually or through containers, on a local machine or on GCP. For this checkpoint, no automation of the installation is expected, nor is it expected to work on the GCP Kubernetes service.

This initial report has a limit of 3 pages, excluding the cover and the table of contents.

Final Report Requirements

The report submitted must contain the following information. Note that the order in which the following points are presented in the report is left to the group's discretion.

- A brief description of the architecture, API and main components of the **Warracker** application. Groups may reuse the text of the initial report.
- Identification of the tools and approach used for the automatic installation and configuration of the application.
- A critical discussion of the questions posed in Task 2.
- Monitoring tools, metrics and visualizations chosen, justifying their choice.
- Evaluation and testing tools developed, justifying their choice.
- Presentation and analysis of the experimental evaluation results.
- Final reflection on the work, pointing out the main strengths and areas for improvement. This reflection must also include a discussion of the deviations observed relative to the originally defined plan (*i.e.*, in the initial report).

The final report does not need to include the provisioning scripts (these are available in the GitHub repository) and should focus on the main decisions made to address the points specified in this assignment description. Although there is no maximum page limit, a report of **10 pages** should be sufficient to address the assignment.

Furthermore, **all members of the group** must be prepared to give a brief presentation of the report, including a 5-minute demonstration. This demonstration must be carried out using Google Cloud and, in order to save time, each group must ensure that the GKE *Kubernetes* cluster is created and usable before the presentation. The group must also ensure that it has enough Google Cloud credits to carry out this demonstration, requesting more credits from the teaching staff if necessary.

Development Requirements

All scripts and configuration files used in carrying out the assignment must be made available through a **GitHub repository** in the **ASCN-UM organization** (<https://github.com/ASCN-UM>). Problems accessing the organization should be reported to the teaching staff.

A **FAQ** will be maintained and updated in the “**Discussions**” section of the ASCN-UM organization, accessible to all students. This FAQ will contain information on:

- The structure of the codebase that each group must use so that we can evaluate it.
- Details on the automatic evaluation of *Checkpoint #2*.
- Access to the Google Cloud platform.

Finally, the teaching staff will provide a set of base Ansible *playbooks* that groups must use. These will be made available later and announced via the *Blackboard* platform.